Housework Credits — A Simple Token Economy on Mandala Chain

Built and deployed step by step, end to end — NC, August 2026

1. What this project is

Housework Credits is a small blockchain-based token economy built as a learning project on Mandala Chain's public testnet. It simulates a household reward system: three accounts (for example Mom, Dad, and Kid) each start with 3 credits, and can send credits to each other to reward completed housework tasks. Every transfer is recorded permanently and verifiably on the blockchain — not in a spreadsheet or a private database that anyone could quietly edit.

The project has two parts: a smart contract (the on-chain rules) and a web frontend (a simple interface non-technical users can click through, connected to their MetaMask wallet).

2. How it works, in plain terms

- Three blockchain accounts are set up in advance (created in MetaMask).
- A smart contract is deployed once, and given the three account addresses and their names.
- The contract automatically gives each account 3 credits when it is deployed.
- Any of the three accounts can send credits to any other of the three accounts, along with a short reason (e.g. "Took out the trash").
- Every transfer is a real blockchain transaction: it costs a small gas fee, gets a unique transaction hash, and is permanently visible on the public block explorer.
- The web page reads the current balances directly from the blockchain every time it loads, so it always shows the real, current state — not a cached or guessed number.

3. The smart contract

The contract is written in Solidity and deployed on Mandala Testnet. It defines three things: who the three accounts are, how many credits each one has, and the rule for sending credits between them.

3.1 The contract code

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.24; contract HouseworkCredits { address[3] public accounts; mapping(address => uint256) public balanceOf; mapping(address => string) public nameOf; event CreditsSent(address indexed from, address indexed to, uint256 amount, string reason); constructor(address[3] memory _accounts, string[3] memory _names) { for (uint256 i = 0; i < 3; i++) { accounts[i] = _accounts[i]; nameOf[_accounts[i]] = _names[i]; balanceOf[_accounts[i]] = 3; } } function sendCredits(address to, uint256 amount, string calldata reason) external
{ require(isKnownAccount(msg.sender), "Sender is not a known account"); require(isKnownAccount(to), "Recipient is not a known account");
require(balanceOf[msg.sender] >= amount, "Not enough credits");
balanceOf[msg.sender] -= amount; balanceOf[to] += amount; emit CreditsSent(msg.sender, to, amount, reason); } function
isKnownAccount(address a) public view returns (bool) { return a == accounts[0] || a == accounts[1] || a == accounts[2]; } function getAllAccounts() external view returns (address[3] memory) { return accounts; } function getAllBalances() external view returns (uint256[3] memory balances) { for (uint256 i = 0; i < 3; i++) { balances[i] = balanceOf[accounts[i]]; } } }
```

3.2 What each part does

- The constructor runs once, at deployment: it records the three account addresses, their names, and gives each one 3 starting credits.
- sendCredits() checks that both sender and recipient are one of the three known accounts, checks the sender has enough credits, then moves the credits and logs an event with the reason given.
- getAllBalances() is a free, read-only function used by the website to display the current balances without spending any gas.

4. Setting up the accounts

Three MetaMask accounts were created in advance, one for each household member. Their public addresses were then passed into the smart contract when it was deployed, along with their display names.

Name	Address
Mom	0x41B369433CBb723DA46b6BA99d737c41e48824A3
Dad	0xb48a48358D4E473BB510F7a45f845018F645821c
Kid	0xed2f24cFd88d5677e34FD5D593416171fE00330e

These three addresses were created as separate MetaMask accounts before deployment.

5. Deploying the contract

The contract was deployed using Foundry, the same toolchain used earlier for the Counter.sol example on Mandala Testnet.

```
forge init housework && cd housework # paste the contract code into src/HouseworkCredits.sol forge create src/HouseworkCredits.sol:HouseworkCredits \ --rpc-url https://rpc1-testnet.mandalachain.io \ --private-key $PRIVATE_KEY \ -- broadcast \ --constructor-args "[0x41B369433CBb723DA46b6BA99d737c41e48824A3,0xb48a48358D4E473BB510F7a45f845018F645821c,0xed2f24cFd88d5677e34FD5D593416171fE00330e]" '["Mom","Dad","Kid"]'
```

Result of the deployment:

Field	Value
Contract address	0xd9A58757F89B936e8C63Be5DEe8eA46120334424
Deployer	0x41B369433CBb723DA46b6BA99d737c41e48824A3 (Mom)
Deployment transaction hash	0x01c2ae1ce0cfb8c193226ac26e20916e62e02ec90fb05eb addb8b88c649ef362

6. The web frontend

A simple webpage was built using plain HTML, CSS, and the ethers.js library, so that someone with no

coding or command-line experience can use the app entirely by clicking buttons.

6.1 What the page does

- Connects to the user's MetaMask wallet.
- Reads the three account names and their current balances directly from the smart contract.
- Displays them as three simple cards.
- Provides a form to send credits: choose a recipient, an amount, and a reason, then click Send.
- Shows a running log of transfers made during the session. ← where?

6.2 Running the page locally

Browser security rules block wallet extensions like MetaMask from working properly on files opened directly from disk. To work around this, the page was served through a simple local web server instead of being opened as a plain file:

```
cd path/to/downloaded/file python3 -m http.server 8080
```

The page was then opened at http://localhost:8080/housework_credits_frontend.html, which allowed MetaMask to connect normally.

7. Testing it end to end

Once the page was connected, a real transfer was made: 1 credit sent from Mom to Dad with the reason "clean home". This produced a real blockchain transaction, confirmed successfully and visible on the public Mandala Testnet explorer, including the exact gas fee paid (a fraction of a KPGT). The balances shown on the page updated immediately afterward to reflect the new state stored on-chain.

AK: screenshot pls

8. Why this matters

This project demonstrates, in a small and understandable form, the same mechanism used by much larger systems: a set of rules (the smart contract) that no single party can secretly alter, values (credits) that are tracked with full transparency, and a simple interface that hides the technical complexity from the end user. The same pattern — accounts, a contract enforcing rules, and a frontend for non-technical use — can be adapted to build other small token-based systems on Mandala Chain.